AI and IoT-Enabled Smart Office System with Cloud-Based Energy Optimization

25-26J-034

Software Requirements Specification (SRS)

B.Sc. (Hons) in Information Technology Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

Names:Shaethayine.R
Student IDs: IT22315700
Supervisor: Prof. Anuradha Jayakody
Date of Submission: 11.1.2026

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to formally define and document the functional and non-functional requirements of the **Intelligent Bandwidth Allocation System**. This document serves as a reference for stakeholders, developers, supervisors, and evaluators to understand the expected behavior, constraints, interfaces, and performance characteristics of the proposed system.This SRS provides a clear and structured description of what the system is expected to do, without detailing how the system will be implemented. It establishes a common understanding of system requirements and acts as a baseline for system development, validation, and future enhancement phases of the project.

1.2 Scope

This document covers the software requirements of an intelligent, machine learning–driven network management system that dynamically allocates bandwidth based on user activity, workload intensity, and real-time network conditions.

The scope of this SRS includes:

- Requirements for collecting and processing software-based user activity and network monitoring data.
- Requirements for Machine Learning models used for task classification, workload classification, bandwidth prediction, and feedback learning.
- Requirements for dynamic bandwidth allocation and enforcement through a simulated or API-based SD-WAN/QoS controller.
- Requirements for monitoring, alerting, logging, and performance evaluation.
- Functional and non-functional requirements related to system reliability, performance, security, and maintainability.

The scope **does not include** hardware-level implementation, embedded system design, or physical IoT sensor integration, as the system is designed to operate using software telemetry and simulated or real network interfaces.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Description
SRS	Software Requirements Specification
ML	Machine Learning
QoS	Quality of Service
SD-WAN	Software Defined Wide Area Network
API	Application Programming Interface
Mbps	Megabits per second
KPI	Key Performance Indicator
Telemetry	Automated collection of system and user activity data
Regression Model	A machine learning model that predicts continuous numerical values
Classification Model	A machine learning model that assigns data to predefined categories
Feedback Learning	Process of improving model behavior using performance outcomes

1.4 References (place this at the end of the document)

Comment: References to all documents that are connected to this one.

1.5 Overview

The Intelligent Bandwidth Allocation System is a software-centric solution designed to optimize network resource utilization in dynamic enterprise environments. The system leverages multiple Machine Learning models to analyze user behavior, identify task types, determine workload intensity, and predict optimal bandwidth allocation in real time.

The system's primary goals are to:

- Improve network performance and fairness through intelligent bandwidth allocation.
- Adapt bandwidth policies dynamically based on user context and workload.
- Reduce network congestion and performance degradation.
- Support continuous learning and system improvement using feedback mechanisms.

The system is intended for use by network administrators, researchers, and academic evaluators who require visibility into bandwidth usage, performance metrics, and allocation decisions. End users interact indirectly with the system through their normal network activities, while administrators monitor and evaluate system behavior through dashboards and logs.

This SRS provides an overview of the system requirements and serves as the foundation for subsequent design, implementation, and validation phases of the project.

2 Overall Descriptions

The Intelligent Bandwidth Allocation System is a software-based system designed to analyze user workload and system activity and dynamically optimize bandwidth allocation using Machine Learning. The system combines a web-based dashboard, multiple ML models, and a backend control layer to provide real-time analysis, prediction, and monitoring. It is designed to be modular and extensible so that future improvements can be introduced without changing the core system structure.

2.1 Product perspective

The proposed system acts as an intelligent optimization layer on top of conventional network monitoring and bandwidth management solutions. Traditional systems mainly rely on static QoS rules or manual configuration by administrators. In contrast, this system uses Machine Learning models to automatically analyze task types, workload intensity, and system behavior to make adaptive bandwidth decisions.

The system can operate independently for analysis and simulation purposes or be logically integrated with SD-WAN or QoS controllers for execution. Compared to existing monitoring tools that only visualize metrics, this system provides prediction, optimization, and feedback-driven improvement.

2.1.1 System interfaces

The system interfaces with the operating system using standard mechanisms:

- File system access to load ML models, datasets, and configuration files
- Runtime execution of Python-based Machine Learning components
- Operating system logging facilities for error and activity logging
- Web server runtime for hosting the dashboard and backend services

No low-level or kernel-level operating system access is required.

2.1.2 User interfaces

The system provides a web-based dashboard accessed via a browser.
Based on the implemented dashboard, the key UI components are:

- **Home Page**: Presents system purpose and core features such as task classification, workload analysis, and issue detection
- **Dashboard Page**:
    - Displays system status (model load status, system errors)
    - Shows analysis statistics (daily analyses, efficiency score)

    - Provides controls to run workload analysis using sample data or user ID
    - Displays recent analyses and historical data sections

The dashboard is designed for clarity and quick interpretation rather than technical configuration.

2.1.3 Hardware interfaces

No special hardware interfaces are required.
The system operates entirely using software-based data sources and does not depend on physical sensors, embedded devices, or specialized networking hardware.

2.1.4 Software interfaces

The system interacts with the following software components:

- **Machine Learning Modules (Python)**
    - Task Classification Model
    - Workload Intensity Classification Model
    - Bandwidth Prediction (Regression) Model
    - Issue Detection / Feedback Model
- **Backend Application**
    - Coordinates data flow between UI and ML models
    - Handles analysis execution and result aggregation
- **Optional Database**
    - Stores historical analysis results and logs
- **Execution Layer (Logical)**
    - SD-WAN / QoS policy interface for bandwidth enforcement (simulated or API-based)

2.1.5 Communication interfaces

The system uses standard communication mechanisms:

- HTTP/HTTPS between frontend dashboard and backend
- REST APIs for triggering analysis and retrieving results
- JSON format for data exchange between system components
- Internal communication between backend services and ML modules

2.1.6 Memory constraints

Memory usage includes:

- Loading trained ML models into memory during execution
- Temporary memory for preprocessing and prediction
- Storage space for logs and optional historical data

The system is designed to operate within normal desktop or server memory limits without requiring specialized hardware.

2.1.7 Operations

Normal operations include:

- Starting the system and loading ML models
- Running workload analysis through the dashboard
- Viewing analysis results, system status, and efficiency metrics
- Monitoring recent and historical analyses

Special operations include:

- Updating or replacing ML models
- Handling model load errors or system failures
- Reviewing logs for debugging and evaluation

2.1.8 Site adaptation requirements

The system can be deployed in:

- Local development environments
- University or laboratory servers
- Cloud-based virtual machines

Configuration values such as model paths, ports, and service settings can be adapted using configuration files or environment variables.

2.2 Product functions

The major functions of the system are:

- Analyze user workload and system activity
- Classify user tasks using Machine Learning
- Determine workload intensity levels
- Predict optimal bandwidth allocation values
- Detect performance issues or inefficiencies
- Present real-time and historical analysis results via dashboard

2.3 User characteristics

The typical users of the system include:

- **Network administrators or IT staff** with basic networking knowledge.

- **Software or system engineers** interested in performance monitoring and optimization.
- **Students and researchers** with general computer literacy.
- **Academic supervisors or evaluators** reviewing system behavior and results.

Users are expected to be familiar with standard web-based applications but do not require expertise in Machine Learning or advanced networking concepts.

2.4 Constraints

The system is subject to the following constraints:

- Dependence on the availability and quality of workload and monitoring data.
- Prediction accuracy is limited by the quality of trained Machine Learning models.
- Real bandwidth enforcement may be simulated in environments without network controllers.
- System performance depends on available computational resources.

2.5 Assumptions and dependencies

The system assumes that:

- Input data reflects realistic workload and network behavior.
- Machine Learning models are trained offline and available for inference.
- Users access the system through supported web browsers.

The system depends on:

- A Python runtime environment for Machine Learning execution.
- A backend server environment for hosting application services.
- Optional database services for persistent data storage.

Future versions of the system are expected to operate on standard desktop or server platforms without major architectural changes.

2.6 Apportioning of requirements

The implementation of system requirements is planned in the following order:

1. Core dashboard and system monitoring functionality.
2. Integration of task classification and workload analysis models.
3. Implementation of bandwidth prediction and optimization logic.
4. Issue detection, logging, and monitoring features.
5. Feedback-driven learning and optimization enhancements.

3 Specific requirements[1] (for Software Dev. Oriented Projects - SRS)

3.1 External interface requirements

3.1.1 User interfaces

The system shall provide a **web-based user interface** implemented using HTML, CSS, and JavaScript and rendered through a Flask backend.

Based on the implemented dashboard in the system, the user interface shall include:

- **Home Page**
  - Display the system name and purpose
  - Highlight main system capabilities such as workload analysis, issue detection, and efficiency monitoring
  - Provide navigation to the dashboard and analysis sections
- **Dashboard Page**
  - Display system status indicators (model availability, system health, database connection status)
  - Show key metrics such as:
    - Number of analyses performed
    - System efficiency score
    - Detected workload or bandwidth issues
  - Provide interactive controls to:
    - Select sample input data or user identifiers
    - Trigger workload and bandwidth analysis
  - Display recent analysis results retrieved from the database
- **Analysis Feedback Elements**
  - Visual indicators for successful analysis execution
  - Error messages when models or database services are unavailable

The user interface is designed for **monitoring and analysis**, not for low-level network configuration. Detailed UI layouts are intentionally kept simple to support clarity and ease of understanding.

3.1.2 Hardware interfaces

The system does not directly interface with any specialized hardware.

- No physical sensors, routers, or embedded devices are required
- Network behavior and bandwidth allocation may be simulated
- The system operates entirely using software-based inputs and monitoring data

3.1.3 Software interfaces

The system shall interface with the following software components:

- **Machine Learning Modules (Python)**

    - Task Classification Model
    - Workload Intensity Classification Model
    - Bandwidth Prediction (Regression) Model
    - Feedback / Issue Detection Model
- **Backend Framework**
    - Flask application (`app.py`) for request handling
    - Model loader and scaler modules
    - MongoDB client interface for data persistence
- **Database System (MongoDB)**
    - Stores workload analysis inputs and results
    - Stores predicted bandwidth values
    - Stores system logs and timestamps
    - Supports retrieval of historical analysis data for dashboard visualization
- **Execution Layer Interface**
    - Logical or simulated SD-WAN / QoS controller
    - Applies bandwidth allocation decisions based on model output

3.1.4 Communication interfaces

The system shall support the following communication mechanisms:

- HTTP/HTTPS communication between web browser and Flask backend
- REST-style API endpoints for:
    - Submitting analysis requests
    - Retrieving prediction results
    - Fetching historical records from MongoDB
- JSON-based data exchange between:
    - Frontend and backend
    - Backend and Machine Learning modules
- Internal communication between backend services and MongoDB using a database driver

3.2 Classes/Objects < For Software Dev. Oriented Projects>

Class / Module	Description	Priority
Web Interface Controller	Handles routing and user requests via Flask	Essential
Data Preprocessing Module	Cleans, scales, and prepares input data	Essential
Task Classification Model	Identifies the type of user task	Essential

Class / Module	Description	Priority
Workload Classification Model	Determines workload intensity level	Essential
Bandwidth Prediction Model	Predicts optimal bandwidth allocation	Essential
Feedback / Issue Detection Model	Analyzes outcomes and detects inefficiencies	Desirable
Execution Controller	Applies bandwidth allocation decisions	Essential
Monitoring Module	Tracks system status and analysis metrics	Essential
Logging Module	Records execution events and errors	Essential
MongoDB Database Handler	Manages storage and retrieval of system data	**Essential**
Model Management Module	Loads and manages ML models	Essential

These classes represent **domain-level components**, not low-level utility classes.

3.3 Performance requirements

The system shall satisfy the following performance requirements:

- Workload analysis and bandwidth prediction shall complete within a response time suitable for interactive dashboard usage
- Machine Learning inference shall execute efficiently without blocking the web interface
- MongoDB read and write operations shall support real-time storage of analysis results
- Dashboard queries for recent and historical data shall be completed within acceptable time limits
- Memory usage shall remain within limits of standard desktop or server systems

3.4 Design constraints

The system design is constrained by the following conditions:

- Machine Learning models must be implemented using Python
- The Flask framework must be used for backend request handling
- MongoDB must be used as the primary database for data persistence

- The system must follow a modular architecture as defined in the system diagram
- No hardware-based data collection is permitted
- Bandwidth enforcement may be simulated during development and evaluation

3.5 Software system attributes

3.5.1 Reliability

The system shall reliably store analysis results in MongoDB and ensure that data is not lost during normal operation. Failures in model execution or database connectivity shall be handled gracefully. The system shall be available whenever the backend server is running and shall support restart without loss of configuration or system integrity.

3.5.2 Availability

The system shall be available whenever the backend server is running and shall support restart without loss of configuration or system integrity. The system shall remain available as long as the Flask server and MongoDB service are operational. The system shall support restart without loss of stored data.

3.5.3 Security

- Access to MongoDB shall be restricted using authentication credentials
- Sensitive configuration details (database URI, credentials) shall be stored securely
- Input data shall be validated before being stored in the database
- Communication between backend and database should follow secure practices
- Access to system functionality shall be controlled through defined interfaces
- Configuration files and model artifacts shall be protected from unauthorized modification
- Input validation shall be enforced to prevent invalid or malicious data processing

3.5.4 Maintainability

- The system shall be modular, allowing individual components to be modified independently
- ML models shall be replaceable without affecting the rest of the system
- Configuration values shall be externalized for easier maintenance
- Database schemas shall be designed to allow future extensions
- MongoDB collections shall support efficient querying and indexing
- ML models and database logic shall remain loosely coupled

- Configuration changes shall not require code modification

3.6 Other requirements

- The system shall maintain logs for all major analysis, prediction, database, and execution events to support monitoring, debugging, and evaluation.
- The system shall store analysis results, predicted bandwidth values, and related metadata with timestamps in the MongoDB database.
- The system shall support retrieval of historical analysis data from MongoDB for visualization and monitoring through the dashboard.
- The database design shall support future analytics, reporting, and trend analysis without requiring major structural changes.
- The system shall provide outputs and visualizations suitable for academic evaluation, demonstrations, and project reviews.
- The system architecture shall support future integration with real SD-WAN or QoS controllers through logical or API-based interfaces.
- The system shall support incremental improvement, retraining, and replacement of Machine Learning models without impacting other system components.
- The system shall allow future migration to cloud-hosted MongoDB services while preserving data integrity and system functionality.
- Logs and monitoring data shall be persistently stored to enable performance evaluation, validation, and future research analysis.